

JavaScript Interview Questions & Answers

52 summarized question-answer pairs, followed by 4 extra important topics with short examples.

Main 52 Questions

Q1. What are the different data types present in JavaScript?

Answer: JavaScript has primitive types: String, Number, BigInt, Boolean, Undefined, Null, and Symbol; and non-primitive type: Object.

Q2. Explain hoisting in JavaScript.

Answer: Hoisting means JavaScript moves variable and function declarations to the top of their scope before execution. Only declarations are hoisted, not initializations.

Q3. Why do we use the word debugger in JavaScript?

Answer: The debugger statement pauses code execution when browser developer tools are open, helping developers inspect variables and debug step by step.

Q4. What is the difference between == and === operators?

Answer: == compares values after type conversion, while === compares both value and type. Example: 2 == '2' is true, but 2 === '2' is false.

Q5. What is the difference between var and let in JavaScript?

Answer: var is function-scoped and can be redeclared; let is block-scoped and helps avoid many hoisting and redeclaration issues.

Q6. Explain implicit type coercion in JavaScript.

Answer: Implicit type coercion is JavaScript's automatic conversion of one data type to another during operations. Example: '5' + 2 gives '52', but '5' - 2 gives 3.

Q7. Is JavaScript statically typed or dynamically typed?

Answer: JavaScript is dynamically typed because variable types are checked at runtime, and the same variable can hold different types of values.

Q8. What is the NaN property in JavaScript?

Answer: NaN means Not-a-Number, but its type is still number. It appears when a numeric operation fails, such as Number('abc').

Q9. Explain passed by value and passed by reference.

Answer: Primitive values are copied by value, while objects are copied by reference-like behavior because variables can point to the same object in memory.

Q10. What is an Immediately Invoked Function Expression (IIFE)?

Answer: An IIFE is a function that runs immediately after it is defined. Example: (function(){ console.log('Hi'); })();

Q11. What is strict mode in JavaScript?

Answer: Strict mode is enabled by 'use strict' and helps catch silent errors, prevents accidental global variables, and makes JavaScript code safer.

Q12. Explain higher-order functions in JavaScript.

Answer: A higher-order function accepts another function as an argument or returns a function. Examples include map(), filter(), and reduce().

Q13. Explain the this keyword.

Answer: this refers to the object that is calling the function. Its value depends on how the function is invoked, not only where it is written.

Q14. What do you mean by self-invoking functions?

Answer: A self-invoking function runs automatically without being called separately, usually by wrapping a function expression and adding ().

Q15. Explain call(), apply(), and bind() methods.

Answer: call() and apply() immediately call a function with a chosen this; apply() passes arguments as an array, while bind() returns a new bound function.

Q16. What is the difference between exec() and test() methods?

Answer: Both are RegExp methods. exec() returns matched result details, while test() returns only true or false.

Q17. What is currying in JavaScript?

Answer: Currying transforms a function with multiple arguments into a chain of functions that take one argument at a time, such as add(2)(3).

Q18. What are some advantages of using external JavaScript?

Answer: External JavaScript improves code readability, reusability, separation of HTML and JS, and team collaboration.

Q19. Explain scope and scope chain in JavaScript.

Answer: Scope controls where variables can be accessed. Scope chain means JavaScript searches for variables from inner scope to outer/global scope.

Q20. Explain closures in JavaScript.

Answer: A closure happens when an inner function remembers variables from its outer function even after the outer function has finished executing.

Q21. Mention some advantages of JavaScript.

Answer: JavaScript is fast, easy to learn, runs on both client and server, and adds interactivity to web pages.

Q22. What are object prototypes?

Answer: Prototype is the mechanism through which JavaScript objects inherit properties and methods from other objects.

Q23. What are callbacks?

Answer: A callback is a function passed as an argument to another function and executed later after a task is completed.

Q24. What are the types of errors in JavaScript?

Answer: Main error types include syntax errors and logical errors. Syntax errors break execution, while logical errors produce wrong output.

Q25. What is memoization?

Answer: Memoization caches function results based on arguments, so repeated calls with the same input return faster without recalculation.

Q26. What is recursion in programming?

Answer: Recursion is when a function calls itself repeatedly until a base condition is met. It is often used in factorial, tree, and sum problems.

Q27. What is the use of a constructor function in JavaScript?

Answer: Constructor functions create multiple similar objects using the new keyword, such as new Person('Ali', 20).

Q28. What is DOM?

Answer: DOM means Document Object Model. It represents HTML/XML as objects so JavaScript can modify page content, structure, and styles.

Q29. Which method is used to retrieve a character from a certain index?

Answer: The `charAt()` method retrieves a character from a specific string index. Example: `'Hello'.charAt(1)` gives `'e'`.

Q30. What do you mean by BOM?

Answer: BOM means Browser Object Model. It allows JavaScript to interact with browser features like window, history, location, screen, and navigator.

Q31. What is the distinction between client-side and server-side JavaScript?

Answer: Client-side JavaScript runs in the browser for UI interaction; server-side JavaScript runs on a server, commonly with Node.js, to handle backend logic.

Q32. What are arrow functions?

Answer: Arrow functions provide shorter syntax and do not bind their own `this`; they inherit this from the surrounding scope.

Q33. What do you mean by prototype design pattern?

Answer: Prototype pattern creates new objects by cloning or using an existing object as a template, which matches JavaScript's prototypal nature.

Q34. What are the differences between declaring variables using `var`, `let`, and `const`?

Answer: `var` is function-scoped, `let` is block-scoped and reassignable, while `const` is block-scoped and cannot be reassigned.

Q35. What is the rest parameter and spread operator?

Answer: Rest collects multiple values into an array in function parameters; spread expands arrays or objects into individual values.

Q36. In JavaScript, how many different methods can you make an object?

Answer: Objects can be created using object literals, constructor functions, classes, `Object.create()`, `new Object()`, or factory functions.

Q37. What is the use of promises in JavaScript?

Answer: Promises handle asynchronous operations and can be pending, fulfilled, or rejected. Results are handled using `then()` and `catch()`.

Q38. What are classes in JavaScript?

Answer: Classes are ES6 syntax for creating constructor-based objects more cleanly. They support constructors, methods, and inheritance.

Q39. What are generator functions?

Answer: Generator functions use `function*` and `yield`. They can pause and resume execution using the `next()` method.

Q40. Explain WeakSet in JavaScript.

Answer: WeakSet stores only objects weakly, so objects can be garbage collected when no other reference exists.

Q41. Why do we use callbacks?

Answer: Callbacks ensure a function runs after another task completes, especially useful for asynchronous operations and event-based programming.

Q42. Explain WeakMap in JavaScript.

Answer: WeakMap stores key-value pairs where keys must be objects and are weakly referenced for garbage collection.

Q43. What is object destructuring?

Answer: Destructuring extracts values from arrays or objects into variables easily. Example: `const { name } = user;`

Q44. What is the difference between prototypal and classical inheritance?

Answer: Classical inheritance uses classes, while prototypal inheritance allows objects to inherit directly from other objects.

Q45. What is a Temporal Dead Zone?

Answer: TDZ is the period between entering a scope and declaring a `let` or `const` variable, where accessing it causes a `ReferenceError`.

Q46. What do you mean by JavaScript design patterns?

Answer: Design patterns are reusable solutions to common coding problems, mainly grouped as creational, structural, and behavioral patterns.

Q47. Is JavaScript a pass-by-reference or pass-by-value language?

Answer: JavaScript passes all arguments by value; for objects, the passed value is a reference copy, so object properties can still be changed.

Q48. Difference between `async/await` and generators usage to achieve the same functionality.

Answer: Generators pause and resume with `yield`, while `async/await` handles promises in cleaner sequential style and always returns a `Promise`.

Q49. What are the primitive data types in JavaScript?

Answer: Primitive data types are basic single-value types: `String`, `Number`, `Boolean`, `Undefined`, `Null`, `Symbol`, and `BigInt`.

Q50. What is the role of deferred scripts in JavaScript?

Answer: `defer` lets HTML parsing continue and runs the script after the document is parsed, improving page loading behavior.

Q51. What has to be done to put lexical scoping into practice?

Answer: A function must keep a reference to the scope chain where it was created, so it can access outer variables later.

Q52. What is the purpose of the given lexical scoping code?

Answer: The code returns an inner function that remembers and returns the local scope value because of closure and lexical environment.

Last 4 Important Questions with Examples

These four topics are added at the end as requested, with question first and answer after it.

Extra Q1. What is the difference between shallow copy and deep copy in JavaScript?

Answer: A shallow copy copies only top-level properties, so nested objects still share references. A deep copy creates a fully independent copy, including nested objects.

```
const user = { name: 'Ali', address: { city: 'Dhaka' } };

const shallow = { ...user };
shallow.address.city = 'Chittagong';
console.log(user.address.city); // Chittagong

const deep = structuredClone(user);
deep.address.city = 'Sylhet';
console.log(user.address.city); // Chittagong, unchanged by deep copy
```

Extra Q2. What are debouncing and throttling? Write a debounce function.

Answer: Debouncing runs a function only after the user stops triggering an event for a delay. Throttling limits a function so it runs at most once within a fixed interval.

```
function debounce(fn, delay) {
  let timer;
  return function (...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}

// Example: call search API after user stops typing.
```

Extra Q3. What are event bubbling, capturing, and delegation?

Answer: Bubbling moves an event from child to parent; capturing moves from parent to child. Delegation uses one parent listener to handle many child elements through event.target.

```
document.querySelector('ul').addEventListener('click', function (e) {
  if (e.target.tagName === 'LI') {
    console.log('Clicked:', e.target.textContent);
  }
});

// One ul listener handles all li clicks.
```

Extra Q4. How does the JavaScript event loop work?

Answer: The event loop coordinates the call stack, Web APIs, microtask queue, and macrotask queue. Promise microtasks run before macrotasks like setTimeout.

```
console.log(1);

setTimeout(() => console.log(2), 0);

Promise.resolve().then(() => console.log(3));

console.log(4);

// Output: 1, 4, 3, 2
```